

claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf\_b531eaeef-39c\agent-af62474f54e9b8740.jsonl`
- SHA-256: `38ae69169a7080d12c72f37c65fc5ccaf2ddc7a638f259ff1a0197d7036a6e8b`
- Source modified: `2026-06-19T17:03:12+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_b531eaeef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

Transcript

1. user (2026-06-19T16:59:09.586Z)

You are an adversarial design reviewer. Be skeptical; the goal is to find real flaws BEFORE we build.

CONTEXT ? badminton/racket-sports rally detector. Pipeline: WASB shuttle DETECTOR (heatmap ? per-frame candidate list) ? single-target online TRACKER (collapses to ONE (x,y) per frame) ? velocity windower ? rallies.

MEASURED (GX010142, owner's anchor clip, on the 21-25s window, from the cached det_raw.jsonl detector candidates):

- WASB's detector emits ≥ 2 candidates on 31% of detected frames (cached, never used past the tracker).
- During the 21-25s rally: 93% of frames carry the ADJACENT-court shuttle (x-norm ~ 0.88 , what the tracker followed),
 - 83% ALSO carry a PROMINENT-court shuttle (x-norm ~ 0.56), 76% carry BOTH at once.
- A greedy nearest-neighbour reconstruction of the prominent-court candidates yields a CONTINUOUS rally arc
 - (75% window coverage, 431px vertical excursion, 8.6px median per-frame jump, ~ 2.4 s sustained)

? VISUALLY CONFIRMED

as a real foreground-court rally (player mid-stroke ? net ? second player), which the single-target tracker discarded.

PROPOSED DESIGN ("the Which problem"):

Split the tracker's two conflated jobs:

- Layer 1 "see everything": a MULTI-TARGET linker over the ALREADY-CACHED det_raw.jsonl candidates ? a SET of shuttle
 - tracks per region (top-K), each with trajectory + duration + motion-energy. Pure-Python over the cache, ZERO GPU,
 - WASB stays frozen (MIT).
- Layer 2 "pick the match's story": score each track by court-membership (the existing prominent-court polygon +
 - point_in_polygon) + rally-likeness (motion energy, vertical excursion, ≥ 2 net-crossings/Gate-A, ends on shuttle-floor-contact) and SELECT the track(s) whose story is the match's; feed those to the windower.

Claimed benefits: recall-POSITIVE (recovers the discarded prominent rally ? a G1 miss) AND precision-positive (drops the adjacent rally) ? unlike today's court_gate which is a subtractive post-filter on the single track (precision-only). Also claimed to SUBSUME the "ghost" (a stuck-pixel false-lock track has ~ 0 motion energy ? never selected). Lands default-OFF, flag-gated, measured on cached candidates + golden labels with no GPU; single-track tracker stays the default.

YOUR LENS:

Attack the ARCHITECTURE/INTEGRATION/REVERSIBILITY. Relocating court-membership UPSTREAM into track-selection: does it cleanly subsume the existing court_gate, Gate-A, and R4, or duplicate/conflict with them? Does adding Layer-1/Layer-2 break the default single-track path (must stay default + a verified no-op when off)? Is a pure-Python multi-target linker over the cache genuinely feasible, or does replacing WASB's WSL tracker drag in WSL/GPU? Where's the cleanest seam? Mitigations.

Return concrete, technically-grounded risks with severity + mitigations. Mark kills_design=true ONLY for a fundamental unmitigable flaw.

2. assistant (2026-06-19T16:59:13.432Z)

I'll investigate the actual code to ground this review in what's really there, rather than trusting the design's claims about clean subsumption.

3. user (2026-06-19T16:59:15.182Z)

```
430de76 docs(rally): capture the shuttle-floor-contact rally-END signal (owner note)
e358b28 feat(eval): prominent-court window gate ? drop multicourt background-court rallies
(default-OFF) (#243)
9736a3b docs: re-ground stale doc claims to the current code (multi-agent audit) (#242)
626061d docs(rally): prominent-court detection design ? kill multicourt background-court FPS
(#241)
3f5a56c fix(gemini): surface failed chunks loudly (no more silent partial reels) (#240)
---
```

```
ABLATION_LAYER.md
ANALYZERS_AND_RECONCILERS.md
BACKLOG.md
CODE_AUDIT_AND_TEST_HARDENING.md
CODE_MAP.md
COMMERCIALIZATION.md
DEFERRED_HARDENING_PLAN.md
DESKTOP_APP_PLAN.md
EVALUATION.md
EXECUTION_POLICY.md
EXPERIMENT_HARNESS.md
FIELD_VISIT_ANSWER_SHEET.md
FIELD_VISIT_PLAYBOOK.md
GOLDEN_DATA_SHARING.md
GOLDEN_REGRESSION_FIXTURES.md
GOLDEN_SET_IMPLEMENTATION.md
GOLDEN_SET_PHASE1_VIDEOS.md
GOLDEN_SET_VENDORING.md
GOLDEN_VIDEOS.md
HARDENING_LOOP_HANDOFF.md
HOSTING_PLAN.md
HOW_RALLY_DETECTION_WORKS.md
I18N_PLAN.md
KHELSUTRA_PRODUCT_OVERVIEW.md
MULTI_SIGNAL_FUSION_PLAN.md
NEXT_STEPS.md
OWNED_MODEL_IMPLEMENTATION_PLAN.md
PERSONALIZATION_PLAN.md
PER_VIDEO_OUTPUT_LAYOUT.md
PLATFORM_ARCHITECTURE.md
PMF_FIELD_SIGNALS.md
PMF_MARKET_RESEARCH.md
POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
PRODUCT_FIELD_ASSOCIATE_FLYER.md
PROJECT_DOC_TEMPLATE.md
PROMINENT_COURT_DETECTION.md
QUALITY_ITERATIONS.md
R1_MILESTONE_LAUNCH_REPORT.md
R2_EVAL_METRIC_DESIGN.md
```

RALLY_DETECTION_GAP_CLOSURE.md
RALLY_DETECTION_QUALITY_REPORT.md
RALLY_QUALITY_RESEARCH.md
README.md
REAL_FOOTAGE_VALIDATION_REPORT.md
ROADMAP.md
ROORA_LABELING_GUIDELINES.md
STORAGE_SHARING_MODEL.md
TRACKNET_WSL_SETUP.md
UI_ALPHA_EXECUTION_PLAN.md
UI_PROPOSAL.md

4. user (2026-06-19T16:59:15.528Z)

backend\pipeline\detectors\rally_gate.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\hardchop\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\boxhill17\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\pr-stitch\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\gap-doc\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\resweep\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\recdoc\backend\pipeline\detectors\rally_gate.py

5. user (2026-06-19T16:59:15.609Z)

backend\pipeline\detectors\court_gate.py
tests\test_court_gate.py

6. assistant (2026-06-19T16:59:19.382Z)

Let me read the core files this design touches: the existing court_gate, rally_gate (Gate-A), and the tracker/windower path.

7. user (2026-06-19T16:59:20.343Z)

```
1      """Prominent-court shuttle gate ? multicourt G2 (see
   ``docs/PROMINENT_COURT_DETECTION.md``).
2
3      In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
   on a
4      background / adjacent court becomes a false rally on the filmed match. This gate re-
   defines the
5      shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
   a visible
6      shuttle point whose normalised position is outside the prominent-court polygon is marked
   NOT
7      visible, so the velocity windower never opens a rally there.
8
9      Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
10     ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
   visibility, never
11     set it ? so on a correct polygon it suppresses out-of-court rallies without
   manufacturing any.
12     ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
   court /
13     unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.
14
15     The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
16     ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
   a point up
17     to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
```

```

still
18     admitted, so a high clear over the prominent court is not clipped.
19     """
20     from __future__ import annotations
21
22     from dataclasses import replace
23     from typing import Any, List, Optional, Sequence, Tuple
24
25     from backend.pipeline.detectors.fusion_features import point_in_polygon
26
27     Polygon = Sequence[Tuple[float, float]]
28
29
30     def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
31         """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?
32
33         The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
34         finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
35         over the prominent court, admit it."""
36         if point_in_polygon((nx, ny), polygon):
37             return True
38         return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)
39
40
41     def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
42                             fps: float, frame_width: float, frame_height: float,
43                             height_pad: float = 0.0) -> float:
44         """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.
45
46         ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
47         visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
48         leave that judgement to the windower)."""
49         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
50             return 1.0
51         fs, fe = interval[0] * fps, interval[1] * fps
52         n = inside = 0
53         for p in points:
54             if getattr(p, "visible", False) and fs <= p.frame <= fe:
55                 n += 1
56                 if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
57                     inside += 1
58         return 1.0 if n == 0 else inside / n
59
60
61     def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
62                                 polygon: Optional[Polygon], fps: float,
63                                 frame_width: float, frame_height: float,
64                                 min_in_court_frac: float = 0.5, height_pad: float = 0.0
65                                 ) -> List[Tuple[float, float]]:
66         """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.
67
68         Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
69         occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
70         window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail

```

```

survives a hard
71         point gate). This drops an interval whose in-court point fraction is below
``min_in_court_frac``,
72         so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
(returns all
73         intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
74         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
75             return intervals
76         return [iv for iv in intervals
77                 if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
78                     >= min_in_court_frac]
79
80
81     def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
82                             frame_width: float, frame_height: float,
83                             height_pad: float = 0.0) -> List[Any]:
84         """Return ``points`` with visibility CLEARED for any visible shuttle outside the
prominent-court
85         polygon (expanded upward by ``height_pad``).
86
87         No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
frame dims
88         are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
list and
89         its points are never mutated)."""
90         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
91             return points
92         out: List[Any] = []
93         for p in points:
94             if getattr(p, "visible", False):
95                 nx, ny = p.x / frame_width, p.y / frame_height
96                 if not admitted(nx, ny, polygon, height_pad):
97                     p = replace(p, visible=False)
98             out.append(p)
99         return out
100

```

8. user (2026-06-19T16:59:20.782Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Optional, Sequence, Tuple
24

```

```

25     Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89

```

```

90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06,
93                       estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94         """Annotate each (start,end) window with its shuttle net-crossing count and a
95         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96         axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98         `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99         `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                   kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

9. assistant (2026-06-19T16:59:25.456Z)

Both gates operate on the post-tracker single trajectory (a `List[points]` with
`.x/.y/.frame/.visible`). Now let me see how they're wired into the pipeline, and find the
tracker, windower, and the proposed prominent-court-window-gate that just landed (#243).

10. user (2026-06-19T16:59:26.352Z)

```

1      # Prominent-Court Detection ? kill background-court false positives (multicourt)
2
3      > **Status:** `DRAFT` ? design + experiment **owner-gated for any default flip** .
**Owner:** Avi . **Created:** 2026-06-19 . **Last updated:** 2026-06-19
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** §8 progress tracker is the source of truth; pointer in memory
`current-state`.
6      > **Relation to existing docs:** attacks gap **G2** ("precision ? local over-fires,
worst on multicourt") from [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md);
it is the **shuttle-side** counterpart of the person-side **Gate B / court mask** in
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md). Complements the held-shuttle
**`shuttle_player_colocation`** (G3) ? court-localization (G2) and held-shuttle (G3) are
independent precision levers.
7      > **Honesty note:** claims are tagged `[verified]` (checked vs code/measured) /
`[design]` (proposed). Every signal lands **flag-gated, default-OFF**, promoted only on measured
lift per [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md) + `[[decision-rigor-norm]]`.
8
9      ---
10
11     ## 0. TL;DR
12     In **multicourt** footage the camera sees several courts. The shuttle detector (WASB)
tracks *any* shuttle in frame, so a rally on a **background / adjacent court** becomes a
predicted rally on the **main match** ? a false positive. The fix: identify the **prominent
court** (the foreground court that occupies the majority of the frame, where the match being

```

filmed is played) and **keep only rallies whose shuttle lives inside it**. Mechanically this re-defines the shuttle signal from `(x, y, visible)` to `(x, y, visible-AND-inside-the-prominent-court)` **before** windowing ? so a background-court shuttle reads as "not there" and never forms a rally. A flat 2D floor polygon is the MVP; a **pseudo-3D play-volume** (court floor + the air column above it) is the refinement so a high clear over the prominent court isn't wrongly rejected. Lands as a **default-OFF experiment**; the flip is gated on a Launch Report ([launch-report-convention]).

13
14 **## 1. Problem & motivating example** `[verified]`
15 - **Measured gap (G2)**: local over-fires ~2x vs Gemini, and the over-fire is **worst on multicourt** ? even among long (>7s) rallies, multicourt over-fires **1.58x** (background-court games are **real** long rallies, just on the wrong court). See `RALLY_DETECTION_GAP_CLOSURE.md` §3` + the P7 long-rally lens.

16 - **Concrete anchor** ? GX010142 rally #1 (2026-06-19). **Owner observation**: "the first rally is just a shuttle flying in the non-prominent court? while the prominent-court players have not started." **Confirmed** from the cached WASB trajectory (`output/wasb/GX010142_1080p__wasb.csv``): rally #1 (`0.72?3.65s``, 2.94s) has a shuttle centroid at **meanX ? 1616** on a 1920-wide frame (?84% to the right) ? spatially apart from the central cluster where most rallies live (meanX ? 1000?1300). Several other suspicious rallies (#3, #9, #12, #16) cluster in the same far-right band ? consistent with a **right-side adjacent court**.

17 - **Why a 1D threshold is not enough** `[verified]` : the courts do not separate cleanly on X or Y alone (rally Y-centroids overlap 250?620 px across courts; the far-right band mixes with foreground play). The courts are **2D regions** under perspective ? so we need the court **geometry**, not a coordinate cutoff. This is the owner's "pseudo-3D / court model" intuition, validated by the data.

18
19 **## 2. Goal & non-goals**
20 - **Goal**: on multicourt clips, drop rallies whose shuttle is outside the prominent court, **without losing real prominent-court rallies** ? measured per-video, no regression on single-court clips (where it must be a no-op or a do-no-harm).
21 - **Non-goals (v1)**: multi-camera; tracking players across courts; full court-line homography for scoring; anything biometric. Single-court footage is explicitly a **no-op** (one court ? everything is "prominent").

22
23 **## 3. Decisions locked**
24 | # | Decision | Rationale |
25 |---|-----|-----|
26 | D1 | The prominent court is a **2D region (polygon)** in image space, refined to a **pseudo-3D play volume** | 1D thresholds don't separate courts under perspective (§1) |
27 | D2 | Gate the **shuttle trajectory** by the region **before windowing** (re-define visibility) | Stops a background rally at the source ? no window ever forms there. Reuses `point_in_polygon`` |
28 | D3 | **Default-OFF**, flag-gated; single-court = no-op | Zero-regression rollout ([weak-signals-retained]); no behaviour change unless a polygon is supplied + the flag set |
29 | D4 | Reuse the existing `court_polygon`` config + `point_in_polygon``; do NOT invent a new geometry stack | `FusionConfig.court_polygon`` + `fusion_features.point_in_polygon`` already exist `[verified]` |
30 | D5 | The prominent court can be **supplied (config)** OR **auto-derived**; auto-derivation is its own measured step | Config polygon is the cheapest first experiment; auto is the product path |

31
32 **## 4. Design**
33
34 **### 4.1 The signal change (the core experiment)**
35 Today the windower consumes the raw WASB trajectory: a per-frame `(Frame, Visibility, X, Y)` . The change is a **pre-windowing filter**:

36 ` `` `
37 ` `` `
38 for each frame f: if shuttle_visible(f) and NOT inside_prominent_court(X_f, Y_f): mark f as NOT visible
39 ` `` `

40
41 A background-court shuttle is then indistinguishable from "no shuttle" ? the velocity windower never opens a rally there. This is purely **subtractive** on the shuttle stream (it can

only remove visibility, never add it), so on a correctly-drawn prominent court it cannot manufacture rallies ? only suppress out-of-court ones. (Mirror of `rally\_gate`'s "do-no-harm" discipline.)

42

43 ### 4.2 Identifying the prominent court (four routes, cheapest first)

44 1. ****Configured polygon**** (MVP, `[design]?buildable now`): a normalized `[[x,y],?]` court polygon per camera/clip, exactly the existing `FusionConfig.court\_polygon`. Draw it once from the dense shuttle/player cluster. Good enough to validate the whole concept on GX010142 + the multicourt golden clips.

45 2. ****Player-location auto-derivation**** (`[design]`): run `PersonDetector.detections\_per\_frame` `[verified]` exists, take the ****largest, most-central, most-persistent**** player cluster (the foreground players are bigger boxes lower in frame), and fit the prominent-court polygon to where **those** players' feet live. "Prominent = where the match's players are." This is why the owner asked to ****extract person features**** ? players anchor the court.

46 3. ****Shuttle-density auto-derivation**** (`[design]`): the prominent court is the densest sustained shuttle-activity region; cluster the visible trajectory and take the dominant mode. Person-free fallback.

47 4. ****Court-line detection**** (`[design]`, heaviest): Hough/segmentation court lines ? the largest court quad. Most accurate, most work; deferred.

48

49 v1 ships route 1 (config) + measures; route 2 (player-anchored) is the product upgrade.

50

51 ### 4.3 Pseudo-3D play volume (the refinement)

52 A flat floor polygon rejects a shuttle at the top of a high clear **over the prominent court** (it projects above the floor quad). The pseudo-3D model treats the prominent court as a ****floor quad + a vertical air column****: accept a shuttle if its image point lies within the floor polygon ****expanded upward**** by a height envelope (the max plausible shuttle height projected through the camera). Practically: the floor quad plus an upward `pad` (perspective-aware ? larger near the net/far baseline). Start with a generous fixed upward pad (cheap, MVP); upgrade to a per-camera height envelope only if measurement shows real rallies being clipped. This keeps the floor tight (rejects adjacent-court floor play) while not clipping legitimate high shuttles.

53

54 ### 4.4 Reuse map `[verified]`

55 | Need | Existing code |

56 |---|---|

57 | point-in-region test |

58 | normalized court polygon config |

59 | player boxes per frame |

60 | trajectory parse + windowing |

61 | per-video scoring | `backend/eval/rally\_seg\_eval.py` (+ the `--stratify` multicourt split, #220) |

62

63 New code is small: a `prominent\_court\_gate(points, polygon, height\_pad)` that filters trajectory points, threaded into the windowing input behind a default-OFF flag.

64

65 ### 4.5 Extension ? deterministic rally-END via shuttle-floor contact (owner note, 2026-06-19) `[design]`

66 The same court geometry that localizes the prominent court also unlocks a ****deterministic rally-END signal**** ? the cheapest, most certain way to know a point is over: ****the shuttle hit the floor.**** This attacks ****G3**** (rally-END over-extension), where today's shuttle-velocity windowing runs the window long because "shuttle still moving ? rally over."

67

68 - ****Introduce two geometric primitives:****

69 - ****The white court lines.**** A badminton court is bounded by ****white, straight lines forming a rectangle**** (with the perspective trapezoid in image space). Detect them ? Hough lines / line-segment detection filtered to white, long, court-consistent segments ? to recover the ****court quad**** (which also **is** the prominent-court floor polygon of §4.2 route 4; one detector serves both).

70 - ****The floor plane.**** The court lines define the ****floor**** (the plane the players

stand on). The shuttle in flight lives **above** this plane; a shuttle **at** the plane (within the trapezoid, or just outside it) is ***on the floor***.

71 - ***The rally-END rule (deterministic):*** a rally ends when the shuttle makes ***floor contact*** ? its trajectory descends to and ***stops/rests at the floor level*** (a near-zero-velocity terminus at the court-line plane), OR a shuttle that goes to the floor ***outside the prominent court*** = an out-of-bounds landing (also point-over). Unlike velocity heuristics, a shuttle resting on the floor is ***unambiguous*** ? the point is over.

72 - ***Why it's strong + cheap:*** no new model, no person stack ? it reuses the WASB trajectory (which already gives the shuttle's descending `(x, y)` + the `Visibility` drop when it settles) plus the court-line geometry. It is the ***shuttle-side*** rally-END signal, complementary to the ***player-state*** rally-END (player kinematics / held-shuttle) in `RALLY\_DETECTION\_GAP\_CLOSURE.md` §4 Lever A ? fuse both for robustness (a shuttle can settle off-camera; a player can pick up early).

73 - ***Subtlety to measure:*** distinguish a true floor landing (point over) from a shuttle that ***dips low but is played back up*** (a tight net shot / low retrieve). The court-line plane + a brief ***dwell*** at floor level (the shuttle rests, doesn't re-accelerate up) is the discriminator; tune the dwell window on golden clips.

74
75 This is tracked as ***C8*** below and cross-links gap-closure ***G3***. It shares the court-line detector with the prominent-court polygon (§4.2 route 4), so the two land together once court-line detection exists.

76

77 ## 5. Experiment plan (measurement)

78 Land the gate flag-gated, then measure on the ***cached trajectories*** (CPU, no GPU/re-detect):

79 1. ***GX010142 sanity:*** draw a prominent-court polygon; confirm ***rally #1 disappears*** and the central rallies survive. The litmus the whole idea is born from.

80 2. ***Multicourt golden*** (`mahadevpura\_2`, GX010128, Badminton_BXH_2, Boxhill_Doubles`): ? rally-count, and once those clips are golden-labelled, ? precision / ? recall vs golden via `rally_seg_eval --stratify` (single vs multicourt). The win condition: multicourt precision up, ***recall flat*** (we drop FPs, not real rallies).

81 3. ***Single-court no-op:*** on single-court clips with no polygon (or a full-frame polygon) the gate is a verified no-op ? assert zero change.

82 4. ***Dual-eval discipline*** ([[eval-dual-set-regression]]): no per-video regression on either eval set.

83

84 Promotion to default-ON is a ***separate, owner-gated*** decision with a Launch Report (both rulers + the over-seg guard), never bundled with the experiment PR.

85

86 ## 6. Risks & limits

87 - ***Polygon is camera-dependent.*** A hand-drawn polygon doesn't generalize across setups ? route 2/3 (auto-derivation) is the real product path; v1 config-polygon is for **validation**.

88 - ***Shuttle height*** (§4.3): too-tight a floor clips high clears; the pseudo-3D pad mitigates, measurement tunes it.

89 - ***The prominent court can change*** (camera re-aim mid-clip): out of scope v1 (assume static per clip).

90 - ***Not a recall fix.*** This only removes out-of-court FPs; it does nothing for missed in-court rallies (G1) ? and must be proven not to **cost** recall.

91

92 ## 7. Open questions (owner / data)

93 1. Auto-derivation anchor: ***player-location*** (route 2) vs ***shuttle-density*** (route 3) as the default once validated? *(decides: data, post-experiment)*

94 2. Height-envelope: fixed upward pad vs per-camera perspective model ? is the fixed pad enough? *(decides: measurement)*

95 3. Do we expose the prominent court as a ***one-time per-camera setup*** (draw-once UI) or always auto? *(decides: owner/product)*

96

97 ## 8. Deliverables & progress tracker ? source of truth

98 Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, off `origin/master`, no stacking.

99

100 | ID | Deliverable | Depends on | Gated? | Status | PR |

101 |----|-----|-----|-----|-----|----|

102 | C0 | This design doc | ? | No | ? | this PR |

103 | C1 | `prominent\_court\_gate(points, polygon, height\_pad)` + unit tests (pure, point-in-

```

polygon) | ? | No (default-OFF) | ? | ? |
104 | C2 | Thread the gate into the windowing input behind a default-OFF flag
(`court_polygon` + `gate_shuttle_to_court`) | C1 | No (default-OFF) | ? | ? |
105 | C3 | GX010142 litmus: rally #1 dropped, central rallies kept (measured) | C2 | No | ?
| ? |
106 | C4 | Extract person features (`detections_per_frame`) on GX010142 ? player-cluster =
prominent court; measure `shuttle_player_colocation` too | ? | No | ? | ? |
107 | C5 | Multicourt-golden measurement (? count now; ? precision/recall vs golden once
labelled) | C2, labels | ? owner labels | ? | ? |
108 | C6 | Auto-derive the polygon (route 2 player-anchored) | C4 | ? owner | ? | ? |
109 | C7 | Launch decision (default-ON) with a Launch Report | C3,C5 | ? owner | ? | ? |
110 | C8 | **Court-line / floor detection** ? (a) auto court polygon, (b) **deterministic
shuttle-floor-contact rally-END** ($4.5, targets G3) | ? | ? owner | ? | ? |
111
112 ## 9. Definition of done
113 - [ ] The gate exists, is **default-OFF**, unit-tested, and a verified no-op on single-
court / no-polygon.
114 - [ ] GX010142 rally #1 (and the far-right adjacent-court band) is dropped while the
central rallies survive `[measured]`.
115 - [ ] On the multicourt golden clips, multicourt **precision improves with recall flat**
vs golden labels (no per-video regression).
116 - [ ] A winning configuration is flipped default-ON only via a Launch Report ? flip
`Status ? DONE`, archive.
117
118 ## 10. References
119 **Code `[verified]`**: `fusion_features.py` (`point_in_polygon`, `person_in_court`),
`person_detector.py` (`detections_per_frame`), `config/models.py`
(`FusionConfig.court_polygon`), `eval/windowing.py`, `eval/rally_seg_eval.py` (`--stratify`).
120 **Docs**: `[RALLY_DETECTION_GAP_CLOSURE.md]` (RALLY_DETECTION_GAP_CLOSURE.md) (G2),
[ `MULTI_SIGNAL_FUSION_PLAN.md` ] (MULTI_SIGNAL_FUSION_PLAN.md) (Gate B / court mask),
[ `EXPERIMENT_HARNESS.md` ] (EXPERIMENT_HARNESS.md),
[ `QUALITY_ITERATIONS.md` ] (QUALITY_ITERATIONS.md).
121
122 ---
123
124 ### Changelog
125 - 2026-06-19 ? Created from the owner's GX010142 rally-#1 (background-court FP)
observation + the measured multicourt over-fire (G2). Validated the 2D/pseudo-3D need from the
cached trajectory; mapped the reuse onto the existing `court_polygon` + `point_in_polygon`
machinery.
126 - 2026-06-19 ? Added $4.5 (owner note): a **deterministic rally-END signal via shuttle-
floor contact** ? detect the white court lines ? the floor plane ? a shuttle resting on the
floor (or landing out-of-court) ends the rally (targets G3). Shares the court-line detector with
the prominent-court polygon; tracked as C8.
127

```

11. user (2026-06-19T16:59:27.038Z)

```

1 # How rally detection works (the 2-layer mental model)
2
3 A short conceptual map of how we go from raw video to rally `[start, end]` segments ?
4 and, crucially, **which layer our annotation can improve.**
5
6 ...
7 video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
8           ?                               ?
9           (WASB CNN,                       per-frame trajectory
10            frozen weights)                 ?
11                                           ?
12                                           velocity signal (how fast the shuttle moves)
13                                           ?
14                                           [SEGMENTATION] ??? the 3 knobs we tune
15                                           ?
16                                           ?
17 rally windows [start, end] ??? IoU vs golden

```

```

labels
18     ...
19
20     ## Two layers ? keep them separate in your head
21
22     **1. DETECTION ? "where is the shuttle, this frame?"**
23     WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a
24     visibility/confidence. Its weights were learned (by gradient descent, on shuttle-
coordinate
25     labels) and we use them **frozen**. We chose a *shuttle* detector on purpose: "shuttle
in
26     flight" is the signal for "a rally is happening," and it's **camera-agnostic** ? unlike
27     player-motion, which fooled the old heuristic on broadcast cameras that pan with the
players.
28     (Your videos are static tripod, so player-motion would also work ? but shuttle-motion is
the
29     robust general bet.)
30
31     **2. SEGMENTATION ? "when is a rally?"**
32     From the per-frame `(x, y)` we compute the shuttle's **velocity** (displacement between
frames).
33     Then three knobs turn that into rally intervals:
34     - **`velocity_thresh`** ? a frame is "active" only if the shuttle is moving faster than
this
35     (in flight, not lying on the floor / being picked up).
36     - **`merge_gap`** ? active frames within this many seconds get joined into one rally
(bridges
37     brief gaps: a shuttle momentarily occluded, or the beat between two hits).
38     - **`min_window_duration`** ? throw away windows shorter than this (blips / false
flickers).
39
40     Output: a list of `[start, end]` windows = the rallies.
41
42     ## Where the annotation fits (the key point)
43     Our Roora/golden labels are **rally `[start, end]` times** ? they say *when* rallies
are, not
44     *where* the shuttle is*. So they can't teach WASB's detector (that needs `(x,y)` labels).
They
45     **can** grade and tune the **segmentation layer**: we score predicted windows against
the golden
46     rallies with **temporal IoU** (how much they overlap), and pick the 3 knobs that
maximise it ?
47     **calibrate on some rallies, measure on held-out ones** so the number is honest.
48
49     ## Where sparks could land (open questions ? not commitments)
50     Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
51     - **Richer features than raw velocity:** acceleration, trajectory *continuity* (a real
rally is a
52     smooth arc that survives occlusion), shuttle *height/arc*, or **direction reversals**
(each
53     reversal ? a shot ? ties to `shots_count`!).
54     - **A tiny learned segmenter** instead of 3 thresholds: feed the trajectory features to
a small
55     model trained on the golden rally labels (still the segmentation layer, just smarter).
56     - **Hysteresis** (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-
rally dips.
57     - **Rhythm/periodicity:** rallies have a hit cadence; dead time doesn't.
58     - **Fusion:** combine shuttle-motion (WASB) with player-motion for static cams. ?
designed in
59     [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (one rally-fusion layer;
the person cue is
60     ADR-007's parked #2 and `rally_gate.py`'s "Gate B" ? now built (default-OFF,
unmeasured on golden) in `fusion_hybrid.py`).
61     - **Per-video calibration as a *feature*:** "watch a few labeled rallies ? auto-tune to
this

```

62 court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right layer.
63
64 The whole point of per-video calibration: a handful of *your* labeled rallies nudges these knobs
65 to *your* court/camera, and we report the IoU lift on the rallies it never saw.
66

12. assistant (2026-06-19T16:59:30.868Z)

Now let me find how court_gate / rally_gate are actually wired into the production pipeline, the tracker, and the windowing entry point. This is the crux of the integration/reversibility critique.

13. user (2026-06-19T16:59:32.406Z)

Found 19 files
tests\test_court_gate.py
backend\eval\rally_seg_eval.py
backend\pipeline\detectors\court_gate.py
tests\test_rally_seg_eval.py
backend\config\models.py
tests\test_served_gate_a_regression.py
tests\test_config.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py
tests\test_personalization_serving.py
backend\eval\fusion_golden.py
tests\test_rally_gate.py
tests\test_fusion_features.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\eval\calibrate_local.py
backend\eval\eval_partial_labels.py
backend\eval\export_wasb_segments.py

14. user (2026-06-19T16:59:32.705Z)

```
1      """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3      **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4      ?
5      ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
6      built
7      on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8      deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9      pipeline
10     (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11     merge_gap=2.0,
12     min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13     is
14     measured on candidates **production never serves**, so a "win" need not transfer. This
15     is
16     exactly how Gate-A scored **0.074** on the proposal set yet **0.008** on the served
17     set
18     (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify`).
19
20     **The fix.** One module owns the two named windowings as :class:`WindowingPreset`, and
21     the
22     SERVED preset is **single-sourced from the same Pydantic config defaults production
23     reads**
24     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
25     Helpers
```

```

16     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17     *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18     operating point a given deployment truly serves.
19
20     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21     windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22     ``per_user_gate``). This module is the what windowing; that one is the promotion
gate**.
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float
61         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63         max_window_duration: float = 0.0
64         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66         hard_cap: bool = False
67         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift

```

```

tail back to
68         #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69         inactivity_end: float = 0.0
70         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71         inactivity_rally_thresh: float = 0.08
72         inactivity_min_density: float = 0.34
73         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76         #: factor keeps it surgical (a normal long rally is left alone).
77         blob_fallback: bool = False
78         blob_factor: float = 4.0
79
80         def as_tuple(self) -> Tuple[float, float, float]:
81             """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82             ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83             bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84             return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86         def to_dict(self) -> Dict[str, Any]:
87             return {
88                 "name": self.name,
89                 "velocity_thresh": self.velocity_thresh,
90                 "merge_gap": self.merge_gap,
91                 "min_window_duration": self.min_window_duration,
92                 "max_window_duration": self.max_window_duration,
93                 "hard_cap": self.hard_cap,
94                 "inactivity_end": self.inactivity_end,
95                 "inactivity_rally_thresh": self.inactivity_rally_thresh,
96                 "inactivity_min_density": self.inactivity_min_density,
97                 "blob_fallback": self.blob_fallback,
98                 "blob_factor": self.blob_factor,
99             }
100
101
102         def _served_preset_from_defaults() -> WindowingPreset:
103             """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105             ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106             - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107             - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108             - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110             Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111             point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112             so the two can never silently diverge again.
113             """
114             idx = IndexingConfig() # construct with defaults ? the served operating point
115             return WindowingPreset(
116                 name="served",
117                 velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
118                 merge_gap=float(idx.dead_time_merge_gap),

```

```

119         min_window_duration=float(idx.min_action_window_duration),
120         max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121         hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)
122         inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
(default 0=OFF)
123         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124         inactivity_min_density=float(idx.inactivity_min_density),
125         blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback
(default OFF)
126         blob_factor=float(idx.window_blob_factor),
127     )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(
158             name=f"served:{family}",
159             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
160             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166             inactivity_min_density=float(idx.get("inactivity_min_density",

```



```

SERVED.inactivity_min_density)),
167         blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168         blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169     )
170
171
172     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173         """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
section
174         (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175         if obj is None:
176             return {}
177         if isinstance(obj, Mapping):
178             sub = obj.get(key, {})
179         elif hasattr(obj, "get"):
180             sub = obj.get(key, {})
181         else:
182             sub = getattr(obj, key, {})
183         return sub if isinstance(sub, Mapping) else {}
184
185
186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                       preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory `points` ? candidate windows under `preset` (the one windowing
call site).
189
190         A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """`[{start, end, ...}]` ? `[(start, end)]` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211

```

15. assistant (2026-06-19T16:59:37.736Z)

This is the key finding: there's an entire eval/serve windowing discipline (`windowing.py`, `promotion.py`) built around the fact that a cue measured on PROPOSAL candidates doesn't transfer to SERVED. The proposed design measures Layer-1/Layer-2 "on cached candidates + golden labels" ? I need to check whether that respects the served-windowing discipline. Let me look at the production segmenter, the tracker, and how court_gate is actually invoked (the #243 PR).

16. user (2026-06-19T16:59:38.938Z)

```

1         """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

```

```

2
3 Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4 TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5 do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6 shuttle-seeded window via two overridable seams the engine exposes:
7
8 - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9 GPU-free, computed from the trajectory we already have); and, only when the person cue
10 is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11 (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12 fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13 torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14 model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15 via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16 - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17 (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18 dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19 **Persisted candidates remain the full superset** regardless, so the offline
experiment
20 harness can re-score any variant.
21
22 Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23 nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24 and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25 segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26 experiment harness (EXPERIMENT_HARNESS.md), never silently.
27 """
28 from typing import Any, Dict, List, Optional, Tuple
29
30 from backend.pipeline.segmenters.base import segmenter_registry
31 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32 from backend.pipeline.detectors.base import DetectorRunner
33 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35 from backend.pipeline.detectors import rally_gate
36 from backend.pipeline.detectors import fusion_features as ff
37
38
39 class FusionSegmenter(TrajectoryHybridSegmenter):
40     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42     family = "fusion"
43     display_label = "Fusion"
44
45     def __init__(self, db: Any, config: Any):
46         super().__init__(db, config)
47         # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48         self._person: Optional[Any] = None
49         # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50         # over the whole trajectory once per candidate window (it depends only on
`points`).
51         self._axis_cache_key: Optional[int] = None
52         self._axis_cache: Optional[tuple] = None

```

```

53
54     @property
55     def name(self) -> str:
56         return "fusion_hybrid"
57
58     @property
59     def description(self) -> str:
60         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                 "AI provider sets semantic boundaries.")
63
64     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66         if substrate == "tracknet":
67             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68             return TrackNetRunner(cfg), {
69                 "weights_path": cfg.weights_path,
70                 "queue_length": cfg.queue_length,
71                 "fusion_substrate": "tracknet",
72             }
73         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                frame_width: float, video_path: str,
78                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                               min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93         # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100     def _axis_estimate(self, points: List[Any]) -> tuple:
101         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so

```

```

it's
102         computed once per video, not once per candidate window (it depends only on
points)."""
103         key = id(points)
104         if self._axis_cache_key != key or self._axis_cache is None:
105             self._axis_cache_key = key
106             self._axis_cache = rally_gate.estimate_play_axis(points)
107         est = self._axis_cache
108         assert est is not None
109         return est
110
111     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                         frame_width: float, video_path: str,
113                         fcfg: Dict[str, Any]) -> Dict[str, float]:
114         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115         frame range and compute the scale/translation-invariant person features. Lazily
builds
116         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118         from backend.pipeline.detectors.person_detector import PersonDetector
119         if self._person is None:
120             self._person = PersonDetector(self.config)
121
122         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127         fw = det["frame_width"] or frame_width
128         fh = det["frame_height"]
129         # If we can't trust the frame dims, every normalised feature would silently
collapse
130         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131         if fw <= 0 or fh <= 0:
132             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134         poly = fcfg.get("court_polygon")
135         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                         court_polygon=polygon, net=net)
140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                           window_stats: List[Dict[str, Any]],
143                           idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
untouched (full superset).
146
147         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
148         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
149         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
150
151         set rather than emit a video with zero rallies. This makes Gate A/B strictly
152         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on

```

```

153         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154         below it always wins on an otherwise-empty video. Observed on real footage
155         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157         fcfg = idx_cfg.get("fusion", {})
158         min_crossings = int(fcfg.get("min_crossings", 0))
159         min_players = int(fcfg.get("min_players", 0))
160         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162         keep: List[int] = []
163         for i, st in enumerate(window_stats):
164             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                 continue
166             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                 continue
168             keep.append(i)
169
170         if not keep and window_stats:
171             # Gating emptied a non-empty video ? fall back to the ungated set.
172             return list(range(len(window_stats)))
173         return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177

```

17. assistant (2026-06-19T16:59:42.733Z)

Now let me look at the base `TrajectoryHybridSegmenter` engine to understand the seam where Layer-1/Layer-2 would integrate, and where court_gate is actually called (#243 says it landed an eval window gate, not production). Let me also find the tracker that does the single-target collapse.

18. user (2026-06-19T16:59:43.460Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27

```

```

28 from backend.pipeline.segmenters.base import BasePlaySegmenter
29 from backend.utils.video import get_video_duration, extract_video_chunk
30 from backend.sports import sport_registry
31 from backend.providers import provider_registry
32 from backend.pipeline.detectors.base import DetectorRunner
33
34 logger = logging.getLogger(__name__)
35
36
37 def _fmt_ts(seconds: float) -> str:
38     seconds = max(0, int(seconds))
39     h, rem = divmod(seconds, 3600)
40     m, s = divmod(rem, 60)
41     return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44 class TrajectoryHybridSegmenter(BasePlaySegmenter):
45     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46 boundaries."""
47
48     #: config section under `indexing` (velocity_threshold lives there), the
49     #: output subdir for trajectories, and the metadata detector-tag prefix.
50     family: str = ""
51     #: human-readable label used in log lines.
52     display_label: str = ""
53
54     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56         """Return (runner, detector-specific detection_params extras)."""
57         raise NotImplementedError
58
59     @staticmethod
60     def _probe_fps_width(video_path: str) -> tuple:
61         """Return (fps, frame_width) for a video, with sensible fallbacks."""
62         cap = cv2.VideoCapture(video_path)
63         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65         cap.release()
66         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68     @staticmethod
69     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70         """Provider-reported exchange count, else a duration-based estimate.
71
72         One canonical implementation ? the twins had drifted (wasb relied on
73         ``int(None)`` raising into the fallback; same result, by accident).
74         """
75         shot_count = segment.get("shuttle_exchange_count")
76         if shot_count is None:
77             return max(3, int(duration / 0.8))
78         try:
79             return int(shot_count)
80         except (ValueError, TypeError):
81             return max(3, int(duration / 0.8))
82
83     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84     -----
85     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86 frame_width: float, video_path: str,
87 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89 BASE
90 returns exactly the 4 motion keys, so the trajectory twins persist byte-
91 identical
92 rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue

```

```

88         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89         return {
90             "peak_velocity": cw["peak_velocity"],
91             "mean_velocity": cw["mean_velocity"],
92             "active_frame_count": cw["active_frame_count"],
93             "track_id_count": cw["track_id_count"],
94         }
95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                           window_stats: List[Dict[str, Any]],
98                           idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                     player_pool: Optional[List[str]] = None,
108                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
123         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
124         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
125         # measure it against the Gemini path, or to avoid the cloud entirely). With
126         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
127         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
128         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
129         if skip_ai:
130             model_name = "local-noai"
131             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
132                       "be emitted as rallies; no %s handoff, no API key needed.",
133                       label, provider_name)
134         else:
135             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
136             api_key_env_var = f"{provider_name.upper()}_API_KEY"
137             api_key = os.environ.get(api_key_env_var)
138             if not api_key:
139                 logger.error(
140                     f"{api_key_env_var} environment variable is not set! "
141                     f"To run the {provider_name} handoff, set your API key. "
142                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
143                 )

```

```

144         return []
145     try:
146         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
147     except KeyError as e:
148         logger.error(str(e))
149         return []
150
151     video_duration = get_video_duration(video_path)
152     if video_duration <= 0:
153         logger.error("Invalid video duration.")
154         return []
155     fps, frame_width = self._probe_fps_width(video_path)
156
157     # --- Runner config + windowing thresholds ---
158     runner, runner_params = self._build_runner(idx_cfg)
159
160     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
163     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
164     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
165     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
166     window_hard_cap = idx_cfg.get("window_hard_cap", False)
167     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
168     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
169     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
170     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
171     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
172     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
173     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
174     log_candidates = idx_cfg.get("log_yolo_candidates", True)
175     store_candidates = idx_cfg.get("store_yolo_candidates", True)
176
177     detection_params = {
178         "detector": self.name,
179         "velocity_thresh": velocity_thresh,
180         "merge_gap": merge_gap,
181         "min_action_window_duration": min_window_duration,
182         **runner_params,
183     }
184
185     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
186     ok, msg = runner.healthcheck()
187     if not ok:
188         logger.error("%s WSL environment not ready: %s", label, msg)
189         return []
190     logger.info("%s WSL env OK: %s", label, msg)
191
192     # --- Phase 2: trajectory -> candidate rally windows ---
193     # Trajectory detectors process the whole video in one pass (they carry
194     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
195     t_start = time.time()
196     out_dir = os.path.join("output", self.family)
197     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
198     if not csv_path:
199         logger.error("%s produced no trajectory; aborting.", label)
200         return []
201
202     points = runner.parse_trajectory_csv(csv_path)
203     logger.info("Parsed %d trajectory points (%d visible).",
204                 len(points), sum(1 for p in points if p.visible))
205
206     all_candidate_windows = runner.trajectory_to_action_windows(

```



```

207         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
208         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
209         min_window_duration=min_window_duration, chunk_index=0,
210         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
211         window_hard_cap=window_hard_cap,
212         window_inactivity_end=window_inactivity_end,
213         inactivity_rally_thresh=inactivity_rally_thresh,
214         inactivity_min_density=inactivity_min_density,
215         window_blob_fallback=window_blob_fallback,
216         window_blob_factor=window_blob_factor,
217     )
218     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
219                 label, time.time() - t_start, len(all_candidate_windows))
220
221     if log_candidates:
222         for cw in all_candidate_windows:
223             logger.info(f"[{label} rally]
224             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
225             f"({cw['end'] - cw['start']:.1f}s) |
226             frames={cw['active_frame_count']} "
227             f"peak_v={cw['peak_velocity']:.3f}
228             mean_v={cw['mean_velocity']:.3f}")
229
230     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
231     fusion
232     # segmenter adds net_crossings + person-cue features. Computed once, reused for
233     both
234     # persistence and the handoff gate below.
235     window_stats = [
236         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
237         idx_cfg)
238         for cw in all_candidate_windows
239     ]
240
241     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
242     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
243     if store_candidates:
244         for i, cw in enumerate(all_candidate_windows):
245             candidate_ids[i] = self.db.add_candidate_segment(
246                 video_id, detector=self.name,
247                 start_time=cw["start"], end_time=cw["end"],
248                 chunk_index=cw["chunk_index"], confidence=min(1.0,
249                 cw["peak_velocity"]),
250                 stats=window_stats[i], detection_params=detection_params,
251                 )
252
253     if not all_candidate_windows:
254         return []
255
256     # --- Phase 3: AI provider handoff over padded candidate clips ---
257     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
258     Persisted
259     # candidates above stay the full superset ? only the served play_segments are
260     gated.
261     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
262     idx_cfg)
263
264     ai_segments: List[dict] = []
265     if skip_ai:
266         # Fully-local: the selected candidate windows ARE the rallies ? emit them
267         verbatim
268         # (no clip extraction, no upload). Boundaries are the local detector's; shot
269         count
270         # falls back to the duration-based estimate (no AI exchange count).
271         ai_segments = [

```

```

260         {
261             "start_time": all_candidate_windows[wi]["start"],
262             "end_time": all_candidate_windows[wi]["end"],
263             "shuttle_exchange_count": None,
264             "shot_count_confidence": "LocalNoAI",
265             "_candidate_id": candidate_ids[wi],
266         }
267         for wi in handoff_indices
268     ]
269     logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
270                 "(no AI handoff).", len(ai_segments))
271 else:
272     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
273     os.makedirs(handoff_dir, exist_ok=True)
274     logger.info("Phase 3: %s validation on %d candidate windows...",
275                 provider_name, len(handoff_indices))
276
277     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
278         w_start = max(0, window["start"] - handoff_padding)
279         w_end = min(video_duration, window["end"] + handoff_padding)
280         w_dur = w_end - w_start
281         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
282         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
283             return []
284         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
285         segments, _ = provider.analyze_video(w_path, prompt,
286                                             label=f"Handoff {wi+1}")
287         valid = []
288         for seg in segments:
289             try:
290                 seg["start_time"] = float(seg["start_time"]) + w_start
291                 seg["end_time"] = float(seg["end_time"]) + w_start
292                 seg["_candidate_id"] = candidate_ids[wi]
293                 valid.append(seg)
294             except (ValueError, TypeError, KeyError) as e:
295                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
296             try:
297                 os.remove(w_path)
298             except OSError:
299                 pass
300         return valid
301
302     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
303         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
304                     for wi in handoff_indices]
305         for future in futures:
306             ai_segments.extend(future.result())
307         try:
308             os.rmdir(handoff_dir)
309         except OSError:
310             pass
311         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
312
313     # --- Phase 4: DB population + candidate linking ---
314     segments_data = []
315     for segment in ai_segments:
316         try:
317             start = float(segment["start_time"])

```

```

318         end = float(segment["end_time"])
319         dur = end - start
320         metadata = {
321             "shot_count": self._shot_count_from(segment, dur),
322             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
323             "detector": f"{self.family}-hybrid-{model_name}",
324         }
325         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
326         if segment_id:
327             candidate_id = segment.get("_candidate_id")
328             if candidate_id is not None:
329                 self.db.link_candidate_to_segment(candidate_id, segment_id)
330             segments_data.append({
331                 "id": segment_id, "start_time": start, "end_time": end,
332                 "duration": dur, "metadata": metadata,
333             })
334         except Exception as e:
335             logger.warning("Skipping invalid segment struct: %s", e)
336
337     return segments_data
338

```

19. assistant (2026-06-19T16:59:47.343Z)

Now I understand the production pipeline shape. The candidate windows come from `runner.trajectory\_to\_action\_windows(points, ...)` where `points` is the single-track post-tracker trajectory parsed from the CSV. Let me check how court_gate is actually wired (the #243 PR was an eval gate, not production) and look at the tracker/det_raw.jsonl to verify the "pure-Python over the cache" claim.

20. user (2026-06-19T16:59:49.391Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py-193-    #
(measured on GX010142 rally #1). No polygon ? no-op. See docs/PROMINENT_COURT_DETECTION.md.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py-194-    poly
= video.get("court_polygon")
backend\eval\rally_seg_eval.py-195-    if poly and ivs:
backend\eval\rally_seg_eval.py:196:        from backend.pipeline.detectors.court_gate import
gate_intervals_to_court
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py-197-
fw = float(video["frame_width"])
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py-198-
fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
backend\eval\rally_seg_eval.py:199:        ivs = gate_intervals_to_court(
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py-200-
ivs, points, poly, float(video["fps"]), fw, fh,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py-201-
min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py-202-
height_pad=float(video.get("court_height_pad", 0.0) or 0.0))

```

21. user (2026-06-19T16:59:50.373Z)

```

Found 30 files limit: 30
backend\pipeline\segmenters\gemini.py
tests\test_ablation.py
tests\test_experiment.py
tests\test_database.py
backend\infrastructure\database.py
backend\eval\golden_fixtures.py
backend\eval\fusion_audio.py
backend\eval\experiment.py

```

```
backend\config\models.py
backend\pipeline\segmenters\yolo_hybrid.py
tests\test_windowing.py
tests\test_fusion_compare.py
tests\test_audio_features.py
scripts\run_phase3_eval.py
scripts\nightly_regression.py
tests\test_golden_real_fixtures.py
tests\test_golden_fixtures.py
tests\test_fusion_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\eval\windowing.py
tests\test_tracknet_hybrid.py
backend\pipeline\detectors\wasb_infer.py
tests\test_wasb_cache.py
backend\pipeline\detectors\base.py
tests\test_reingest_safety.py
tests\test_serve_contrast.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py
backend\eval\served_gate_a.py
```

22. assistant (2026-06-19T16:59:58.398Z)

Confirmed: court_gate is currently ONLY wired into the eval path (`rally\_seg\_eval.py`), NOT production. The design claims it "subsumes the existing court_gate" ? but court_gate isn't in production yet either. Let me check the WASB runner to see how the single-target trajectory is produced and whether det_raw.jsonl (the multi-candidate cache) actually exists and is detached from the WSL/GPU path.

23. user (2026-06-19T16:59:59.617Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
```

```

33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
37     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
38     """
39     import os
40     import os.path as osp
41     import csv
42     import glob
43     import json
44     import argparse
45     import logging
46     from collections import defaultdict
47     from contextlib import contextmanager
48
49     logger = logging.getLogger(__name__)
50
51     CACHE_VERSION = 1
52     _DET_FILE = "det_raw.jsonl"
53     _MANIFEST_FILE = "manifest.json"
54
55
56     def _build_cfg(weights: str, sport: str):
57         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
58         from hydra import compose, initialize
59         with initialize(config_path="configs", version_base=None):
60             cfg = compose(config_name="eval", overrides=[
61                 f"dataset={sport}",
62                 "model=wasb",
63                 f"detector.model_path={weights}",
64                 "runner.device=cuda",
65                 "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
66             ])
67         return cfg
68
69
70     def _frame_id(path: str) -> int:
71         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
72         stem = osp.splitext(osp.basename(path))[0]
73         digits = "".join(ch for ch in stem if ch.isdigit())
74         return int(digits) if digits else -1
75
76
77     # ----- #
78     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79     # ----- #
80     def _cache_paths(cache_dir: str):
81         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84     def default_cache_dir(out_csv: str) -> str:
85         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86         d = osp.dirname(osp.abspath(out_csv))
87         stem = osp.splitext(osp.basename(out_csv))[0]
88         return osp.join(d, stem + "_wasbcache")
89
90
91     def _atomic_write_text(path: str, text: str) -> None:
92         """Write then rename so a crash mid-write can't leave a half-written file."""
93         tmp = path + ".tmp"

```

```

94         with open(tmp, "w") as f:
95             f.write(text)
96             f.flush()
97             os.fsync(f.fileno())
98         os.replace(tmp, path)
99
100
101     def _det_plain(det) -> list:
102         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
103         list."""
104         xy = det["xy"]
105         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
106
107     def _dets_to_tracker(plain_list):
108         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
109         array,
110         the tracker does vector math / np.linalg.norm on it)."""
111         import numpy as np
112         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
113                 for p in plain_list]
114
115     def write_manifest(cache_dir: str, manifest: dict) -> None:
116         _, mpath = _cache_paths(cache_dir)
117         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120     def read_manifest(cache_dir: str):
121         _, mpath = _cache_paths(cache_dir)
122         if not osp.exists(mpath):
123             return None
124         try:
125             with open(mpath) as f:
126                 return json.load(f)
127         except (json.JSONDecodeError, OSError):
128             return None
129
130
131     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                             frames_in: int, frames_total: int) -> bool:
133         """A cache is reusable only if the run that produced it matches this run's
134         inputs."""
135         if not manifest or manifest.get("version") != CACHE_VERSION:
136             return False
137         return (
138             manifest.get("frames_dir") == osp.abspath(frames_dir)
139             and manifest.get("weights") == weights
140             and manifest.get("sport") == sport
141             and manifest.get("frames_in") == frames_in
142             and manifest.get("frames_total") == frames_total
143         )
144
145     def reset_cache(cache_dir: str) -> None:
146         """Drop any existing cache so the next run starts clean."""
147         det_jsonl, mpath = _cache_paths(cache_dir)
148         for p in (det_jsonl, mpath):
149             if osp.exists(p):
150                 os.remove(p)
151
152
153     def load_and_clean_cache(cache_dir: str):
154         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155         rebuild the per-frame detection accumulation, and rewrite the file to exactly that

```

```

156     prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159     [x, y, score, scale] candidates (accumulated across every window the frame is in,
160     in window order ? identical to a non-resumed run).
161     """
162     det_jsonl, _ = _cache_paths(cache_dir)
163     det_by_fid = defaultdict(list)
164     valid: list = []
165     if osp.exists(det_jsonl):
166         with open(det_jsonl, "r") as f:
167             for line in f:
168                 s = line.strip()
169                 if not s:
170                     continue
171                 try:
172                     obj = json.loads(s)
173                 except json.JSONDecodeError:
174                     break # truncated trailing line (crash mid-
flush)
175                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                     break
177                 valid.append(s)
178                 for fid, plain in obj["f"]:
179                     det_by_fid[int(fid)].extend([list(p) for p in plain])
180                 # Guarantee a clean append boundary by rewriting only the good prefix.
181                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182     return len(valid), det_by_fid
183
184
185 def _append_windows(fh, objs) -> None:
186     """Append window records as JSONL and durably flush (bounded loss on crash)."""
187     for o in objs:
188         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189     fh.flush()
190     os.fsync(fh.fileno())
191
192
193 def _atomic_write_trajectory(out_csv: str, rows) -> None:
194     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
195     tmp = out_csv + ".tmp"
196     with open(tmp, "w", newline="") as f:
197         w = csv.writer(f)
198         w.writerow(["Frame", "Visibility", "X", "Y"])
199         w.writerows(rows)
200         f.flush()
201         os.fsync(f.fileno())
202     os.replace(tmp, out_csv)
203
204
205 # ----- #
206 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207 # ----- #
208 _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211 def synthetic_frame_path(index: int) -> str:
212     """Stable, index-encoding frame name used by the streaming path.
213
214     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
215     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216     frames came from disk or from the in-memory stream. The streaming image loader
217     inverts this name back to an index to fetch the decoded frame.
218     """
219     return _STREAM_FRAME_FMT.format(int(index))

```

```

220
221
222 def build_windows(frames: list, frames_in: int):
223     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224     + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226     Returns a list of windows, each a list of ``frames_in`` consecutive paths
227     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230     """
231     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234 class SequentialFrameStore:
235     """In-memory sliding buffer over a forward-only frame reader, sized for the
236     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
237
238     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
243     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
244     earliest index any not-yet-finished window can still ask for) and decode each source
245     frame exactly once via the forward-only ``reader(index)``.
246     """
247
248     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249         self._reader = reader
250         self._total = int(total)
251         self._window = max(1, int(window))
252         self._buf: dict = {}
253         # On a resumed run the detector's first needed frame is `start_index`; begin
254         # pulling there so the reader can skip (seek past) the already-cached prefix
255         # instead of decoding 0..start_index-1 just to throw them away.
256         self._next = int(start_index) # next index not yet pulled from the reader
257         self._max_seen = int(start_index) - 1
258         self._floor = int(start_index) # lowest index we still keep (never evict
below)
259
260     def get(self, index: int):
261         index = int(index)
262         if index < self._floor:
263             raise KeyError(
264                 f"frame {index} already evicted (below floor {self._floor}); the access
"
265                 f"pattern must stay within a window of the highest frame seen")
266         if index >= self._total:
267             raise KeyError(f"frame {index} out of range (total {self._total})")
268         # Pull forward from the reader until the requested index is buffered.
269         while self._next <= index:
270             self._buf[self._next] = self._reader(self._next)
271             self._next += 1
272         if index > self._max_seen:
273             self._max_seen = index
274         # Evict frames older than the earliest a still-running window could re-request:
275         # once we've seen index `m`, no future window starts before `m - (window-1)`.
276         new_floor = max(self._floor, self._max_seen - (self._window - 1))
277         for k in range(self._floor, new_floor):
278             self._buf.pop(k, None)
279         self._floor = new_floor
280         return self._buf[index]
281
282

```



```

283     def _index_from_synthetic_path(path: str) -> int:
284         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287         in lockstep with the on-disk naming scheme.
288         """
289         return _frame_id(path)
290
291
292     def _bgr_to_pil_rgb(frame_bgr):
293         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
294         PIL RGB image WASB's disk path produces.
295
296         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
297         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
298         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
299         frame indistinguishable from the on-disk one to everything downstream.
300         """
301         import cv2
302         from PIL import Image
303         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
304
305
306     def _make_streamed_read_image(store, real_read_image):
307         """Build the ``read_image`` replacement used during a streaming detector pass.
308
309         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
310         matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
311         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
312         without the WSL-only ``dataloaders`` package.
313         """
314         def _read_image(path, *args, **kwargs):
315             idx = _index_from_synthetic_path(path)
316             if idx < 0:
317                 return real_read_image(path, *args, **kwargs)
318             return _bgr_to_pil_rgb(store.get(idx))
319         return _read_image
320
321
322     @contextmanager
323     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
324         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
325         frames during the detector pass, byte-for-byte as it would over real PNGs.
326
327         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
328         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
329         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
330         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
byte-identical to the disk round-trip, and the shim never touches the filesystem so
331         the
332         ``osp.exists`` guard is sidestepped for synthetic stream paths.
333
334         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually

```

```

335     calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
336     module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
337     reference).
338
339     ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
340     in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
341     resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
342     original reader is restored on exit.
343     """
344     from dataloaders import dataset_loader as _dl
345     store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
346     real_read_image = _dl.read_image
347     # Rebind read_image in the consuming module for the duration of the detector pass;
348     # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
349     _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
350     try:
351         yield store
352     finally:
353         _dl.read_image = real_read_image # type: ignore[assignment]
354
355
356     # ----- #
357     # Inference
358     # ----- #
359     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
360           limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361           log_every_batches: int = 50, cache_dir: "str | None" = None,
362           flush_every: int = 1000, fresh: bool = False,
363           frame_loader=None, source_key: "str | None" = None) -> str:
364         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366         Two equivalent ways to supply pixels (output is bit-identical between them):
367
368         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
370         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
371           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
372           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
373           reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
374           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
375           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
376           ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
``num_workers=0``
377           (the in-process shim + buffer can't cross worker processes).
378
379           ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
380           frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
381           after reboot still matches.
382           """
383     import time
384     import torch
385     from torch.utils.data import DataLoader
386     from detectors import build_detector
387     from trackers import build_tracker

```

```

388 from dataloaders import build_img_transforms, build_seq_transforms
389 from dataloaders.dataset_loader import ImageDataset
390 from utils import Center
391
392 streaming = frame_loader is not None
393
394 cfg = _build_cfg(weights, sport)
395 frames_in = int(cfg["model"]["frames_in"])
396 input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
397 output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
398
399 if streaming:
400     # `frames` is already the ordered list of (synthetic) frame paths.
401     if not isinstance(frames, (list, tuple)):
402         raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
403     frames = list(frames)
404     frames_dir = source_key or "stream"
405 else:
406     frames_dir = frames
407     frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
408     if limit:
409         frames = frames[:limit]
410     if len(frames) < frames_in:
411         where = frames_dir if not streaming else "video stream"
412         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
413
414     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
415     samples = []
416     for window in build_windows(frames, frames_in):
417         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
418         samples.append({"frames": window, "annos": annos})
419     windows_total = len(samples)
420
421     # ---- cache setup / resume decision ----- #
422     if cache_dir is None:
423         cache_dir = default_cache_dir(out_csv)
424         os.makedirs(cache_dir, exist_ok=True)
425         det_jsonl, _ = _cache_paths(cache_dir)
426         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427         manifest = None if fresh else read_manifest(cache_dir)
428         resume = manifest_compatible(
429             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430             frames_in=frames_in, frames_total=len(frames),
431         )
432     if resume:
433         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435     else:
436         if manifest is not None:
437             logger.info("cache incompatible with this run -> starting fresh")
438             reset_cache(cache_dir)
439             windows_done, det_by_fid = 0, defaultdict(list)
440
441     base_manifest = {
442         "version": CACHE_VERSION,
443         "frames_dir": cache_key,
444         "weights": weights,
445         "sport": sport,
446         "frames_in": frames_in,
447         "frames_total": len(frames),

```

```

448         "windows_total": windows_total,
449         "windows_done": windows_done,
450     }
451     write_manifest(cache_dir, base_manifest)
452
453     # ---- detector pass over the un-cached windows ----- #
454     remaining = samples[windows_done:]
455     if remaining:
456         _, transform_test = build_img_transforms(cfg)
457         try:
458             _, seq_transform_test = build_seq_transforms(cfg)
459         except Exception:
460             seq_transform_test = None
461         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462                          transform=transform_test, seq_transform=seq_transform_test,
463                          is_train=False)
464         # Streaming forces single-process loading: the in-memory frame store + the
465         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
466         workers.
467         loader_workers = 0 if streaming else num_workers
468         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
469                             num_workers=loader_workers)
470         detector = build_detector(cfg)
471
472         from contextlib import ExitStack
473
474         t0 = time.time()
475         done = windows_done
476         win_idx = windows_done
477         log_every = max(1, log_every_batches)
478         flush_n = max(1, flush_every)
479         pending = []
480         logger.info(f"detecting on {len(remaining)} remaining windows "
481                    f"(total {windows_total}, batch={batch_size},
482                    f"source={'video-stream' if streaming else 'frames-dir'})...")
483         det_f = open(det_jsonl, "a")
484         try:
485             with ExitStack() as stack:
486                 stack.enter_context(torch.no_grad())
487                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
488                 # to the
489                 # in-memory decoded frame for that synthetic path; every other line of
490                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
491                 # exactly.
492                 # The detector's first needed frame is `windows_done` (window i starts
493                 # at
494                 # frame i), so prime the store there for a resume.
495                 if streaming:
496                     stack.enter_context(
497                         _streaming_read_image_patch(frame_loader, len(frames),
498                                                      window=frames_in,
499                                                      start_index=windows_done))
500                 for bi, batch in enumerate(loader):
501                     imgs, _hms, trans = batch[0], batch[1], batch[2]
502                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
503                     path
504                     batch_results, _ = detector.run_tensor(imgs, trans)
505                     nb = imgs.shape[0]
506                     for ib in range(nb):
507                         frames_payload = []
508                         for ie in sorted(batch_results[ib].keys()):
509                             p = img_paths[ie][ib]
510                             fid = _frame_id(p)
511                             plain = [_det_plain(d) for d in batch_results[ib][ie]]

```

```

504         frames_payload.append([fid, plain])
505         det_by_fid[fid].extend(plain)
506         pending.append({"w": win_idx, "f": frames_payload})
507         win_idx += 1
508     done += nb
509     if len(pending) >= flush_n or done >= windows_total:
510         _append_windows(det_f, pending)
511         pending = []
512         base_manifest["windows_done"] = done
513         write_manifest(cache_dir, base_manifest)
514     if (bi + 1) % log_every == 0 or done >= windows_total:
515         el = time.time() - t0
516         new_done = done - windows_done
517         rate = new_done / el if el > 0 else 0.0
518         eta = (windows_total - done) / rate if rate > 0 else 0.0
519         logger.info(f"{done}/{windows_total} "
520                    f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                    f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522     if pending:
523         _append_windows(det_f, pending)
524         base_manifest["windows_done"] = done
525         write_manifest(cache_dir, base_manifest)
526     finally:
527         det_f.close()
528         logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529 else:
530     logger.info("detector cache already complete -> tracker-only re-run")
531
532 # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533 tracker = build_tracker(cfg)
534 tracker.refresh()
535 rows = []
536 for fid in sorted(det_by_fid.keys()):
537     r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538     rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540 _atomic_write_trajectory(out_csv, rows)
541 visible = sum(1 for _, v, *_ in rows if v)
542 logger.info(f"wrote {len(rows)} rows -> {out_csv}")
543 logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544 return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:
548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:

```

```

568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578     def _probe_frame_count(video_path: str) -> int:
579         """Container-reported frame count via cv2 (0 if unknown). Used to size the
580         stream."""
581         import cv2
582         cap = cv2.VideoCapture(video_path)
583         if not cap.isOpened():
584             raise SystemExit(f"cannot open video: {video_path}")
585         n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586         cap.release()
587         return n
588
589     def make_video_frame_loader(video_path: str):
590         """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
591
592         ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
593         long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594         detector's sliding-window order); it reads sequentially and only uses a frame-seek
595         to
596         skip forward when a *resume* starts past the current position. Each ``cap.read()``
597         returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
598         would
599         yield (PNG is lossless), so the detector sees identical pixels to the disk path.
600
601         Returns the count hint from the container so the caller can build the frame list;
602         ``release`` closes the capture.
603         """
604         import cv2
605         cap = cv2.VideoCapture(video_path)
606         if not cap.isOpened():
607             raise SystemExit(f"cannot open video: {video_path}")
608         count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
609         state = {"pos": 0} # next index cap.read() will return
610
611         def frame_loader(index: int):
612             index = int(index)
613             if index < state["pos"]:
614                 raise RuntimeError(
615                     f"streaming decode is forward-only; got index {index} after
616                     {state['pos']}")
617             if index > state["pos"]:
618                 # Forward skip only happens when resuming past cached windows. Seek once.
619                 cap.set(cv2.CAP_PROP_POS_FRAMES, index)
620                 state["pos"] = index
621                 ok, frame = cap.read()
622                 if not ok:
623                     raise RuntimeError(f"failed to decode frame {index} from {video_path}")
624                 state["pos"] += 1
625                 return frame
626
627         def release():
628             cap.release()
629
630     return frame_loader, count_hint, release

```

```

629
630     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                             limit: int = 0, batch_size: int = 8,
632                             log_every_batches: int = 50, cache_dir: "str | None" = None,
633                             flush_every: int = 1000, fresh: bool = False) -> str:
634         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637         ``run``'s docstring for why.
638         """
639         frame_loader, count_hint, release = make_video_frame_loader(video_path)
640         try:
641             if count_hint <= 0:
642                 raise SystemExit(
643                     f"could not determine frame count for {video_path}; cannot stream "
644                     f"(fall back to frame extraction with --frames_dir)")
645             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646             source_key = "video:" + osp.abspath(video_path)
647             return run(frame_list, weights, out_csv, sport, limit,
648                       batch_size=batch_size, num_workers=0,
649                       log_every_batches=log_every_batches, cache_dir=cache_dir,
650                       flush_every=flush_every, fresh=fresh,
651                       frame_loader=frame_loader, source_key=source_key)
652         finally:
653             release()
654
655     def main():
656         ap = argparse.ArgumentParser()
657         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
658         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
659         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
660         ap.add_argument("--stream-video", action="store_true",
661                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
662
663                         "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664         ap.add_argument("--weights", required=True)
665         ap.add_argument("--out", required=True)
666         ap.add_argument("--sport", default="badminton")
667         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
668         ap.add_argument("--batch-size", type=int, default=8)
669         ap.add_argument("--num-workers", type=int, default=4)
670         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671         ap.add_argument("--cache-dir", default=None,
672                         help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673         ap.add_argument("--flush-every", type=int, default=1000,
674                         help="fsync the detector cache every N windows (bounds reboot
loss)")
675         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676         args = ap.parse_args()
677
678         if args.stream_video:
679             if not args.video:
680                 raise SystemExit("--stream-video requires --video")
681             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                                batch_size=args.batch_size,
683                                log_every_batches=args.log_every_batches,

```

```

cache_dir=args.cache_dir,
684             flush_every=args.flush_every, fresh=args.fresh)
685         return
686
687     frames_dir = args.frames_dir
688     if args.video:
689         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690     if not frames_dir:
691         raise SystemExit("provide --frames_dir or --video")
692     run(frames_dir, args.weights, args.out, args.sport, args.limit,
693         batch_size=args.batch_size, num_workers=args.num_workers,
694         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695         flush_every=args.flush_every, fresh=args.fresh)
696
697
698 if __name__ == "__main__":
699     main()
700

```

24. assistant (2026-06-19T17:00:04.832Z)

Critical finding. The `det\_raw.jsonl` cache lives INSIDE WSL (`~/models/WASB-SBDT/src/`), produced by `wasb\_infer.py` which runs in the WSL conda env. The detector candidates are written there next to the output. The tracker pass (`tracker.update`) ? which does the single-target collapse ? ALSO runs in WSL because it imports WASB's `trackers` module and does numpy vector math. Let me verify where the cache physically ends up relative to the Windows host, and confirm the parse path. Let me check the wasb_runner adapter and whether det_raw.jsonl is retrievable on the host.

25. user (2026-06-19T17:00:05.804Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24    logger = logging.getLogger(__name__)
25
26    # Windows path to the inference wrapper we stage into WSL.
27    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30    @dataclass
31    class WasbConfig:

```



```

32     """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33     distro: str = "Ubuntu"
34     repo_dir: str = "~/models/WASB-SBDT"
35     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36     conda_env: str = "wasb"
37     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38     sport: str = "badminton"
39     wsl_stage_dir: str = "~/clips"
40     timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41     keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42     min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43     # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44     # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45     # extraction that dominates a long clip. Output is bit-identical to the PNG path
46     # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47     stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                  f"python -c \"import torch; print('torch', torch.__version__, "
76                  f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)

```

```

91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104 res.stdout).strip())
105             return None
106         return dest
107
108 def _stage_infer_script(self) -> bool:
109     """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110     src_mnt = to_wsl_mnt_path(_INFER_WIN)
111     cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112 OK"
113     res = self._wsl_bash(cmd)
114     return res.returncode == 0 and "OK" in (res.stdout or "")
115
116 def run_predict(self, video_win_path: str, output_win_dir: str,
117                 video_id: Optional[str] = None) -> Optional[str]:
118     """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120     All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121     two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122     can never reuse each other's staged frames, resumable cache, or CSV.
123
124     """
125     # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126     # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127     # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
128     # the CSV landed inside WSL while Windows polled the repo-relative path.
129     output_win_dir = os.path.abspath(output_win_dir)
130     os.makedirs(output_win_dir, exist_ok=True)
131     # Normalize for cross-platform basename (tests use Windows paths on Linux).
132     video_win_path = str(video_win_path).replace("\\", "/")
133     base = os.path.basename(video_win_path)
134     stem, ext = os.path.splitext(base)
135     # Unique, content-derived key: same filename + different bytes -> different key.
136     key = f"{stem}__{video_id[:12]}" if video_id else stem
137     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
138     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
139
140     if not self._stage_infer_script():
141         logger.error("Could not stage wasb_infer.py into WSL.")
142         return None
143     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
144     if staged_video is None:
145         return None
146     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
147
148     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
149     # WSL stage filesystem is already low so the user can clean up before it fills.
150     # (Streaming never materializes the PNGs, so the guard only matters off the fast
151     path.)
152
153     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
154         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
155         if free is not None and free < self.cfg.min_free_gb:
156             logger.warning(
157                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame

```

```

extraction "
152             "may fill the disk ? consider running tools/cleanup_caches.py.",
153             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157         if self.cfg.stream_video:
158             frame_args = "--stream-video "
159         else:
160             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161         cmd = (
162             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163             f"python wasb_infer.py "
164             f"--video {wsl_tilde_quote(staged_video)} "
165             f"{frame_args}"
166             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167             f"--sport {shlex.quote(self.cfg.sport)} "
168             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv'}}"
169         )
170         logger.info("Running WASB inference on %s ...", base)
171         try:
172             res = self._wsl_bash(cmd)
173         except subprocess.TimeoutExpired:
174             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175             return None
176         if res.returncode != 0:
177             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178             return None
179         if not os.path.exists(expected_csv_win):
180             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181             return None
182         logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184         # Success ? the trajectory CSV is the durable output; the staged video copy and
185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

26. assistant (2026-06-19T17:00:12.703Z)

This is decisive for the WSL/GPU critique. Key facts:

1. The trajectory CSV is written to `wsl\_out\_dir` which is `to\_wsl\_mnt\_path(output\_win\_dir)` ? i.e. it lands on the Windows host via `/mnt/`. But the `det\_raw.jsonl` cache lives in `default\_cache\_dir(out\_csv)` = `<out>\_wasbcache` next to the CSV. Since the CSV out path is the

`/mnt/` Windows path, the cache DOES land on the host. Good for the design's claim.

2. BUT: on ****success****, `run\_predict` deletes the staged video + frame cache. Let me check whether it preserves the `\_wasbcache` dir. It only removes `[frames\_dir, staged\_video]` ? NOT the `\_wasbcache`. So `det\_raw.jsonl` survives. But the design assumes it's reliably present; let me verify the cache hygiene/cleanup tooling doesn't purge it, and confirm the candidate format. Let me also check the tracker collapse and the rally_seg_eval integration point.

27. user (2026-06-19T17:00:14.124Z)

```
150         with open(manifest_path) as fh:
151             manifest = json.load(fh)
152         out: List[Dict[str, Any]] = []
153         for v in manifest:
154             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
155             gts: List[Interval] = []
156             if os.path.exists(feat):
157                 with open(feat) as fh:
158                     gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
159 [])]
159             entry: Dict[str, Any] = {"name": v["name"], "trajectory": v["trajectory"],
160                                     "fps": float(v["fps"]), "frame_width":
161 float(v["frame_width"]),
162                                     "gts": gts}
163             # Optional prominent-court gate inputs (multicourt G2; default absent ? the gate
164 is a no-op):
165             # a normalized court_polygon + optional frame_height + court_height_pad from the
166 manifest.
167             for k in ("court_polygon", "frame_height", "court_height_pad"):
168                 if k in v:
169                     entry[k] = v[k]
170             out.append(entry)
171         return out
172
173     def windows_for(video: Dict[str, Any], preset: WindowingPreset,
174                     min_crossings: int = 0) -> List[Interval]:
175         """Parse a video's trajectory, window it under `preset` ? predicted rally
176 intervals.
177         ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
178         ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
179 this many
180 times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
181 derived
182 from the trajectory + an auto-estimated net axis; it cleans up the over-split that
183 bounded
184 windowing (W1) leaves behind.
185         NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
186 would
187 drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
188 is
189 strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
190 harm".
191         """
192         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
193         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
194         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
195         ivs = windowing.windows_to_intervals(wins)
196         # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
197 normalized
198         # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the
199 prominent court ? a
200         # background/adjacent-court rally. Window-level (not point-level): a background
```

```

rally's central
192     # tail keeps a few in-court points, enough to hold a window, so point-clearing alone
is too leaky
193     # (measured on GX010142 rally #1). No polygon ? no-op. See
docs/PROMINENT_COURT_DETECTION.md.
194     poly = video.get("court_polygon")
195     if poly and ivs:
196         from backend.pipeline.detectors.court_gate import gate_intervals_to_court
197         fw = float(video["frame_width"])
198         fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
199         ivs = gate_intervals_to_court(
200             ivs, points, poly, float(video["fps"]), fw, fh,
201             min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
202             height_pad=float(video.get("court_height_pad", 0.0) or 0.0))
203     if min_crossings > 0 and ivs:
204         from backend.pipeline.detectors.rally_gate import gate_windows
205         gated = [(g.start, g.end)
206                 for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
207                 if g.kept]
208         # Gating emptied a non-empty video ? keep the ungated windows.
209         ivs = gated if gated else ivs
210     return ivs
211
212
213     #: One windowed video before scoring: (name, court_type, predicted intervals, GT
intervals).
214     PredictedVideo = Tuple[str, str, List[Interval], List[Interval]]
215
216
217     def predict_all(golden: List[Dict[str, Any]], preset: WindowingPreset,
218                    min_crossings: int = 0) -> List["PredictedVideo"]:
219         """Window every scorable golden video ONCE (the expensive trajectory-parse +
windowing step),
220         returning ``(name, court_type, preds, gts)`` per video. Lets callers re-score the
same
221         predictions under several lenses (e.g. the all-vs-long stratified report) without
re-windowing."""
222         out: List[PredictedVideo] = []
223         for v in golden:
224             if not v["gts"] or not os.path.exists(v["trajectory"]):
225                 continue
226             out.append((v["name"], court_type_for(v), windows_for(v, preset, min_crossings),
v["gts"]))
227         return out
228
229
230     def _score_predicted(predicted: List["PredictedVideo"], iou: float,
231                          tau_start: Optional[float], tau_end: Optional[float],
232                          min_duration: float) -> List[Dict[str, Any]]:
233         """Score pre-windowed videos into per-video rows (carrying ``name`` +
``court_type``)."""
234         rows: List[Dict[str, Any]] = []
235         for name, court, preds, gts in predicted:
236             row = score_intervals(preds, gts, iou, tau_start, tau_end, min_duration)
237             row["name"] = name
238             row["court_type"] = court
239             rows.append(row)

```

28. user (2026-06-19T17:00:15.263Z)

```

tools\cleanup_caches.py:15:    python -m tools.cleanup_caches --include-wasbcache # also drop
detector resume caches
tools\cleanup_caches.py:35:# protects it. wasbcache is regenerable but default_purge=False (it
enables resume and

```

```

tools\cleanup_caches.py:36:# is tiny) ? opt in with --include-wasbcache.
tools\cleanup_caches.py:46:         if low.endswith("_wasbcache"):
tools\cleanup_caches.py:47:             return ("detector resume cache", True, False) # opt-in
via --include-wasbcache
tools\cleanup_caches.py:86:             include_wasbcache: bool, now: float) ->
Tuple[List[Entry], List[Entry]]:
tools\cleanup_caches.py:93:         purge = default_purge or (include_wasbcache and cat ==
"detector resume cache")
tools\cleanup_caches.py:232:     ap.add_argument("--include-wasbcache", action="store_true",
help="also purge detector resume caches")
tools\cleanup_caches.py:247:         include_wasbcache=args.include_wasbcache, now=now)
tests\test_cleanup_caches.py:23:     # wasbcache is regenerable but opt-in only (default_purge
False)
tests\test_cleanup_caches.py:24:     cat, regen, default_purge =
classify("adarsh_traj_wasbcache", True, "wsl")
tests\test_cleanup_caches.py:33: def test_plan_default_purges_frames_keeps_csv_and_wasbcache():
tests\test_cleanup_caches.py:38:     _e("v_traj_wasbcache", 500, True),
tests\test_cleanup_caches.py:41:             include_wasbcache=False,
now=1_000_000.0)
tests\test_cleanup_caches.py:44:     assert {e.name for e in kept} == {"v_traj.csv",
"v_traj_wasbcache"}
tests\test_cleanup_caches.py:47: def test_plan_include_wasbcache_opts_it_in():
tests\test_cleanup_caches.py:48:     entries = [_e("v_traj_wasbcache", 500, True)]
tests\test_cleanup_caches.py:50:             include_wasbcache=True,
now=1_000_000.0)
tests\test_cleanup_caches.py:51:     assert [e.name for e in to_del] == ["v_traj_wasbcache"]
tests\test_cleanup_caches.py:57:             include_wasbcache=False,
now=1_000_000.0)
tests\test_cleanup_caches.py:69:             include_wasbcache=False,
now=now)
backend\pipeline\detectors\wasb_infer.py:22:     ``<cache>/det_raw.jsonl`` (flushed+fsync'd every
``--flush-every`` windows) and
backend\pipeline\detectors\wasb_infer.py:31:The cache is keyed by the output path
(``<out>_wasbcache/`` by default), lives next
backend\pipeline\detectors\wasb_infer.py:52:_DET_FILE = "det_raw.jsonl"
backend\pipeline\detectors\wasb_infer.py:88:     return osp.join(d, stem + "_wasbcache")
backend\pipeline\detectors\wasb_infer.py:154:     """"Read the longest *contiguous* (w == line
index) valid prefix of det_raw.jsonl,
backend\pipeline\detectors\wasb_infer.py:672:             help="resumable detector cache
dir (default: <out>_wasbcache next to --out)")
tests\test_wasb_cache.py:165:     assert osp.basename(cd) == "sample_long_traj_wasbcache"

```

29. assistant (2026-06-19T17:00:25.558Z)

This is a crucial integration finding. The `det\_raw.jsonl` cache:

- Is opt-in purgeable via `--include-wasbcache`, default-kept (good), but the cache-hygiene policy in memory explicitly purges intermediates. It's regenerable ONLY via a full GPU re-run.
- The candidates in `det\_raw.jsonl` are in WASB's **model output coordinate space** (`out\_width`/`out\_height`, e.g. 512x288 heatmap space) via `\_det\_plain` ? NOT the original-video pixel space. The tracker upscales. The court polygon + windowing operate in original-frame pixel space (frame_width). So a Layer-1 linker over the cache must replicate WASB's coordinate transform ? a hidden coupling. Let me verify the coordinate space of the cached candidates vs the CSV by checking the tracker output transform.

Let me look at the tracker build and how the candidate `xy` maps to CSV `X,Y`, plus check the WASB tracker source to assess whether a pure-Python reimplementation is faithful.

30. user (2026-06-19T17:00:29.038Z)

not-on-host

```

Adarsh v Avi__bb961d0d0cff_wasb_wasbcache
GX010125_1080p__89605e80ed01_wasb.csv
GX010125_1080p__89605e80ed01_wasb_wasbcache
GX010142_1080p__c4cfa94a792f_wasb.csv

```

```
GX010142_1080p__c4cfa94a792f_wasb_wasbcache
GX010143_1080p__c991e53106c8_wasb.csv
GX010143_1080p__c991e53106c8_wasb_wasbcache
GX010144_1080p__74affb36e15e_wasb.csv
GX010144_1080p__74affb36e15e_wasb_wasbcache
GX010145_1080p__5a6fe8c350d4_wasb.csv
GX010145_1080p__5a6fe8c350d4_wasb_wasbcache
GX010147_1080p__79ae625eac6d_wasb.csv
GX010147_1080p__79ae625eac6d_wasb_wasbcache
mahadevpura_smoke_180s__05e0e577029d_wasb.csv
mahadevpura_smoke_180s__05e0e577029d_wasb_wasbcache
smoke_GX010143__1803d4c01a73_wasb.csv
smoke_GX010143__1803d4c01a73_wasb_wasbcache
```

31. user (2026-06-19T17:00:30.157Z)

```
Found 14 files
backend\pipeline\segmenters\gemini.py
backend\providers\gemini.py
backend\config\models.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\wasb_infer.py
backend\eval\fusion_golden.py
backend\eval\score_calibration.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\fusion_features.py
backend\utils\video.py
backend\pipeline\validators\relevance.py
backend\eval\rally_seq_proto.py
backend\eval\gemini_refine.py
```

32. assistant (2026-06-19T17:00:34.153Z)

The cached `\_wasbcache` dirs are present on the host next to the CSVs. Let me inspect the actual GX010142 cache to confirm the coordinate space of cached candidates and verify the design's measured claims are reproducible, then check the WASB tracker's coordinate transform (the gap a Layer-1 reimplementation must replicate).

33. user (2026-06-19T17:00:35.952Z)

```
total 3189
drwxr-xr-x 1 avidu 197609      0 Jun 19 14:10 .
drwxr-xr-x 1 avidu 197609      0 Jun 19 17:27 ..
-rw-r--r-- 1 avidu 197609 3250556 Jun 19 14:10 det_raw.jsonl
-rw-r--r-- 1 avidu 197609    310 Jun 19 14:10 manifest.json
=== manifest ===
{
  "version": 1,
  "frames_dir": "video:/home/avidullu/clips/GX010142_1080p__c4cfa94a792f.mp4",
  "weights": "/home/avidullu/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
  "sport": "badminton",
  "frames_in": 3,
  "frames_total": 29937,
  "windows_total": 29935,
  "windows_done": 29935
}=== first det line ===
{"w":0,"f":[[0,[]],[1,[]],[2,[]]]}
{"w":1,"f":[[1,[]],[2,[]],[3,[]]]}
{"w":2,"f":[[2,[]],[3,[]],[4,[]]]}
{"w":3,"f":[[3,[]],[4,[]],[5,[]]]}
{"w":4,"f":[[4,[]],[5,[]],[6,[]]]}
{"w":5,"f":[[5,[]],[6,[]],[7,[]]]}
{"w":6,"f":[[6,[]],[7,[]],[8,[]]]}
```

```

{"w":7,"f":[[7,[]],[8,[]],[9,[]]]}
{"w":8,"f":[[8,[]],[9,[]],[10,[]]]}
{"w":9,"f":[[9,[]],[10,[]],[11,[]]]}
{"w":10,"f":[[10,[]],[11,[]],[12,[]]]}
{"w":11,"f":[[11,[]],[12,[]],[13,[]]]}
{"w":12,"f":[[12,[]],[13,[]],[14,[]]]}
{"w":13,"f":[[13,[]],[14,[]],[15,[]]]}
{"w":14,"f":[[14,[]],[15,[]],[16,[]]]}
{"w":15,"f":[[15,[]],[16,[]],[17,[]]]}
{"w":16,"f":[

```

34. assistant (2026-06-19T17:00:40.386Z)

The cache exists, with the windowed structure (each frame appears in up to 3 windows ? the `frames\_in=3` overlap). Note the duplicate-frame accumulation: `det\_by\_fid[fid].extend(plain)` accumulates a frame's candidates across all 3 windows it appears in, so a Layer-1 linker reading the raw cache must dedupe per-frame or it triple-counts. Let me find a frame with actual candidates to confirm the coordinate space, and check the relevant frames around the 21-25s window (21*~30fps ? frame 630).

35. user (2026-06-19T17:00:44.291Z)

```
frames-with-candidates(window-counted): 28104 with>=2: 2375
sample frame in 600-800: None
```

36. user (2026-06-19T17:00:50.594Z)

```

X range: 425.84229469299316 1916.25
Y range: 5.199126899242401 791.7280197143555
multi frame 51 [[1684.5685958862305, 178.5007381439209, 13.868836402893066, 0],
[1862.2479629516602, 586.2407112121582, 7.4761786460876465, 0]]
```

37. assistant (2026-06-19T17:00:57.402Z)

Important: the cached candidate coordinates are in **original-frame pixel space** (X up to 1916 on a 1920-wide frame, Y up to ~792 ? wait, 792 not 1080; this is the model letterboxing ? but X reaches ~1920). So the candidate `xy` is largely in source-pixel space already (WASB's detector returns post-scale coords). This means a Layer-1 linker can operate in the same coordinate space as `court\_polygon` (normalized by frame_width/height) ? but the Y ceiling at ~792 (not 1080) suggests a non-trivial scale on Y that the design must verify, or the 1080p frames are letterboxed/cropped. This is a real coordinate-fidelity risk.

Now let me verify the claim that the tracker is the single-target collapse, and check what `track\_id\_count` means in the windower (the design says the windower consumes one (x,y)/frame). Let me look at the windower in tracknet_runner.

38. user (2026-06-19T17:00:59.210Z)

```

backend\pipeline\detectors\tracknet_runner.py:6:    1. Translate the Windows video path to its
WSL-visible form (and optionally
backend\pipeline\detectors\tracknet_runner.py:80:class TrajectoryPoint:
backend\pipeline\detectors\tracknet_runner.py:82:    visible: bool
backend\pipeline\detectors\tracknet_runner.py:98:    """Verify the WSL env is usable: conda
env exists, TF imports, GPU visible.
backend\pipeline\detectors\tracknet_runner.py:237:    def parse_trajectory_csv(csv_win_path:
str) -> List[TrajectoryPoint]:
backend\pipeline\detectors\tracknet_runner.py:246:    visible=int(row["Visibility"]) == 1,
backend\pipeline\detectors\tracknet_runner.py:260:    Computes each visible point's
normalised velocity from the last *visible* point;
backend\pipeline\detectors\tracknet_runner.py:267:    if not p.visible:
backend\pipeline\detectors\tracknet_runner.py:281:    def trajectory_to_action_windows(
backend\pipeline\detectors\tracknet_runner.py:301:    A frame is "active" when the shuttle
is visible AND its inter-frame

```


backend\pipeline\detectors\tracknet_runner.py:355:
shuttle; kept for window-shape parity

"track_id_count": 1, # single

39. user (2026-06-19T17:01:03.282Z)

```
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):
251                     continue
252         points.sort(key=lambda p: p.frame)
253         return points
254
255     @staticmethod
256     def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
257                       velocity_thresh: float) -> List[Tuple[int, float]]:
258         """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
259
260         Computes each visible point's normalised velocity from the last *visible* point;
261         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263         the caller so identical floats feed the velocity math."""
264         active = [] # (frame_idx, velocity)
265         prev: Optional[TrajectoryPoint] = None
266         for p in points:
267             if not p.visible:
268                 prev = None # break the trajectory; absent shuttle = not in flight
269                 continue
270             if prev is not None:
271                 dframes = p.frame - prev.frame
272                 if dframes > 0:
273                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                     velocity = (dist / frame_width) * (fps / dframes)
275                     if velocity > velocity_thresh:
276                         active.append((p.frame, velocity))
277             prev = p
278         return active
279
280     @staticmethod
281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,
```

```

294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
309         the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
310         and a single window swallows several rallies + the dead time between them. When
311         window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
312         longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
313         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
314         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
315         drop, and it is config-gated default-OFF until measured on the golden set.
316
317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
318         over-long window has NO internal inactivity gap to split on (a continuously-
active
319         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
320         blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
322         (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
325         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
326         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         """
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:

```

```

338         frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]

```

40. assistant (2026-06-19T17:01:08.661Z)

I now have a complete picture of the architecture. The windower hardcodes `track\_id\_count: 1` ? single shuttle is baked into the velocity windower's data contract. The entire downstream (velocity, windowing, Gate-A, court_gate, R4) operates on a flat `List[TrajectoryPoint]` (one point per frame).

Let me do two final checks: (1) confirm the design's reconstruction claim is reproducible from the cache (is the prominent-court arc really there?), and (2) check the config model to see how flags/court_polygon are currently structured, to assess the integration seam cleanliness.

41. user (2026-06-19T17:01:16.558Z)

```

frames-with-det in 21-25s: 0
adjacent(x>0.78): 0 (0%) prominent(0.45-0.70): 0 (0%) both: 0 (0%)

```

42. assistant (2026-06-19T17:01:20.213Z)

Zero detections in frames 630-750? The fps assumption may be wrong (GoPro is often ~59.94 or ~50fps). 29937 frames is suspicious. Let me check the actual fps and re-probe.

43. user (2026-06-19T17:01:23.792Z)

```

total frames-with-det: 11750 min 42 max 29936
fps=30.0: frames 21-25s [630-750] with det = 0
fps=50.0: frames 21-25s [1050-1250] with det = 35
fps=59.94: frames 21-25s [1258-1498] with det = 217

```

44. assistant (2026-06-19T17:01:27.070Z)

At 59.94fps, frames 1258-1498 carry detections. The clip is ~59.94fps GoPro (29937/59.94 ? 500s ? 8.3min). Let me reproduce the multi-candidate / prominent / adjacent claim at the correct fps.

45. user (2026-06-19T17:01:32.959Z)

detected frames in 21-25s: 217
>=2 candidates: 150 (69%)
adjacent(x>0.80): 195 (89%)
prominent(0.45-0.68): 112 (51%)
both at once: 98 (45%)
OVERALL >=2 candidate rate: 920/11750 = 7%

46. assistant (2026-06-19T17:01:39.931Z)

The directional claims reproduce (adjacent-court shuttle dominant ~89%, a prominent-court shuttle co-present ~51%, both-at-once ~45%, multi-candidate frames common in this window ~69%) ? but my quick numbers differ from the brief's (93%/83%/76%, 31% overall). The discrepancy is explained by methodology: the brief's "31% of detected frames" is almost certainly ****window-counted**** (not deduped ? the cache triple-counts each frame across 3 windows), and the prominent/adjacent x-band thresholds differ from my coarse cuts. The signal is real and present in the cache; the exact percentages are method-sensitive. That itself is a finding: the design's headline numbers are sensitive to a dedup/counting decision that isn't specified.

I have enough grounding. Let me verify one last integration point ? the config model structure for flags ? then synthesize.

47. user (2026-06-19T17:01:41.772Z)

```
backend\config\models.py:84:class FusionConfig(BaseModel):
backend\config\models.py:88:    shuttle net-crossing count (Gate A signal) and ? only when
`cue_flags['person']` is
backend\config\models.py:91:    is optional: `court_polygon=None` ? Gate B counts every
detection (player-count-only);
backend\config\models.py:102:    cue_flags: dict[str, bool] = Field(default_factory=dict)
backend\config\models.py:110:    court_polygon: Optional[list[list[float]]] = None
backend\config\models.py:117:class IndexingConfig(BaseModel):
```

48. assistant (2026-06-19T17:01:55.629Z)

I have a thorough, code-grounded picture now. Let me synthesize the adversarial architecture/integration/reversibility review.

Key facts established from the code:

1. ****court_gate, Gate-A, R4 are NOT in production**** ? court_gate is wired only into `rally\_seg\_eval.py` (eval); Gate-A's served path is default-OFF (`min\_crossings=0`); R4 (`window\_inactivity\_end`) defaults 0=OFF. So "subsume the existing court_gate" subsumes an ***eval-only*** artifact, not a production filter.
2. All three operate on the ****flat post-tracker `List[TrajectoryPoint]\*\*** (one (x,y)/frame). The windower hardcodes `track_id_count: 1`. The data contract everywhere downstream is single-track.
3. The `det\_raw.jsonl` cache ****does land on the Windows host**** (next to the CSV via `/mnt/`), is mostly source-pixel coords, and is pure JSON ? a pure-Python linker is genuinely feasible without WSL/GPU for ***re-linking***. BUT regenerating the cache for a ***new*** video still requires the full WSL/GPU detector pass, and the WASB ***tracker*** (the single-target collapse) runs in WSL with numpy + WASB's `trackers` module.
4. The cache triple-counts frames (frames_in=3 overlap); coords have a Y ceiling ~792 not 1080 (scale/letterbox subtlety); the brief's headline percentages are sensitive to dedup/counting choices not specified.
5. The eval/serve windowing discipline (`windowing.py` + `promotion.py`) exists precisely because cues measured on PROPOSAL candidates don't transfer to SERVED. A Layer-1/Layer-2 selection measured "on cached candidates" must respect this or it repeats the Gate-A +0.074/?0.008 trap.

49. user (2026-06-19T17:02:56.446Z)

Structured output provided successfully

50. assistant (2026-06-19T17:03:12.713Z)

I've completed the adversarial architecture/integration/reversibility review, grounded in the actual code and the cached detector candidates.

Verdict: ****design-needs-revision**** ? the core idea (split "see everything" from "pick the story") is architecturally sound and the recovered prominent-court signal is genuinely present in `det\_raw.jsonl`, but several integration claims don't survive contact with the code.

Key code-grounded findings:

1. ****No-op seam is achievable but the contract is single-track.**** Everything downstream (``tracknet_runner.py::trajectory_to_action_windows`/`_active_frames`, `rally_gate`, `court_gate`, R4) consumes a flat `List[TrajectoryPoint]` ? one (x,y)/frame ? and the windower hardcodes `track_id_count: 1`. The clean seam is a single pre-windowing substitution that returns that same shape; Layer-2 must SELECT one winning track and emit it in legacy shape. K>1 track merge semantics are the load-bearing unaddressed decision (doubles / simultaneous rallies).`
2. ****"Subsumes court_gate" subsumes an eval-only artifact.**** ``court_gate`` is wired ONLY into ``backend/eval/rally_seg_eval.py::windows_for`` (lines 194-202), not production. Gate-A served path is default-OFF (``min_crossings=0``), R4 (``window_inactivity_end``) defaults 0=OFF. If court-membership moves into Layer-2 while court_gate stays in the scorer, you double-gate during the very measurement meant to justify the design.
3. ****"Measured on cached candidates" risks the eval?serve trap**** that ``backend/eval/windowing.py`` + ``promotion.py`` were built to prevent ? exactly the Gate-A +0.074(PROPOSAL)/?0.008(SERVED) split. The recovered arc must be re-confirmed at the SERVED windowing.
4. ****Pure-Python over the cache is feasible but caveated.**** ``det_raw.jsonl`` does land on the Windows host (verified present for GX010142) and is plain JSON, so re-linking is GPU-free. BUT: a new video needs the full WSL/GPU pass first; the cache triple-counts frames (`frames_in=3`); coords have a Y-ceiling ~792 on a 1080 frame (scale/letterbox to replicate). The brief's headline percentages are dedup-sensitive ? I reproduced ~7% multi-candidate overall (deduped) vs the brief's 31% (likely window-counted); the directional claim (adjacent dominant ~89%, prominent co-present ~51%, both ~45% in-window) holds.

No ``kills_design=true`` ? every risk has a concrete mitigation. Relevant files:

```
`C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\tracknet_runner.py`, `...\court_gate.py`,  
`...\rally_gate.py`, `...\wasb_infer.py`, `...\wasb_runner.py`,  
`...\segmenters\trajectory_hybrid.py`, `...\segmenters\fusion_hybrid.py`,  
`...\eval\windowing.py`, `...\eval\rally_seg_eval.py`.
```